

5.7 ESP32 BLE Bluetooth Web Chat

ESP32 BLE Bluetooth Web Chat

What You'll Build

After completing this project, you'll have a **ESP32 + Computer Webpage** Bluetooth two-way chat system:

- ▶  **One ESP32 development board:** Acts as a BLE Bluetooth peripheral, advertising and waiting for connections
- ▶  **Computer web page:** Opens in Chrome / Edge, connects to ESP32 via Web Bluetooth API
- ▶  **Two-way chat:**
 - ▶ Web page input → displayed on ESP32 serial monitor
 - ▶ ESP32 serial input → displayed on web page
- ▶  **LED command control:** Buttons on the web page (or input commands) can remotely turn on / off the ESP32 onboard LED
- ▶  **Onboard LED:** GPIO2 onboard LED controlled by `/ledon` and `/ledoff` commands from the web page

Simple analogy: It's like chatting with a device via WeChat Web — the web page is the chat window, the ESP32 is the "person" on the other end. You send messages, it receives them; it sends messages, you see them; and you can remotely control its light.



 **Difference from the dual-board Bluetooth version:** The dual-board version requires two ESP32 boards to communicate with each other; this project only needs **one ESP32 + a computer browser**, and the web page talks directly to the ESP32 via Bluetooth.

Step 1: Preparation (Hardware + Software)

Hardware Checklist

Device	Description
ESP32 development board × 1	Must support BLE (ESP32 / ESP32-WROOM / ESP32-S3 all work)
USB data cable × 1	For code flashing and serial communication
Computer × 1	Needs Bluetooth module (laptops almost always have one)

Software Checklist

Software	Description
Arduino IDE	Used to compile and flash ESP32 code
Chrome or Edge browser	 Must use Chrome or Edge — Firefox and Safari do not support Web Bluetooth API



 **Important!** Web Bluetooth API is only supported in Chrome, Edge (Chromium-based), and Opera. Firefox and Safari are not supported. Chrome for Android on mobile is also supported.

Communication Flow Description

1. **ESP32 starts up:** Initializes BLE, starts advertising under the name `ESP32_BLE_Chat`
2. **Web page connects:** Open the web page in a browser, click "Connect Bluetooth", select `ESP32_BLE_Chat`
3. **GATT connection established:** Web page connects to ESP32's BLE service via Web Bluetooth API
4. **Two-way chat:**
 - ▶ Web page input → written to ESP32 via BLE → displayed on ESP32 serial monitor
 - ▶ ESP32 serial input → notified to web page via BLE → displayed on web page
5. **LED control:**
 - ▶ Web page clicks "Turn LED On" button → sends `/ledon` → ESP32 lights up GPIO2 LED
 - ▶ Web page clicks "Turn LED Off" button → sends `/ledoff` → ESP32 turns off LED

Step 3: Software Configuration and Code Flashing



Step 3.1: Open the Project Code

1. Copy the entire project folder to a location on your computer you can find (e.g., desktop)
2. You'll see the following structure:

- [ESP32_Bluetooth_Chat/ESP32_Bluetooth_Chat.ino](#) → ESP32 flashing code
- [web/index.html](#) → Computer Bluetooth web page

Step 3.2: Modify Configuration (Optional)



💡 **Default configuration works out of the box!** If you want to customize the Bluetooth name, just modify the ESP32 code.

Change ESP32 Bluetooth name ([ESP32_Bluetooth_Chat.ino](#)):

```
1 | BLEDevice::init("ESP32_BLE_Chat"); // ⚠ Change to your desired Bluetooth name
```



⚠ If you change the Bluetooth name, just select the modified name when the web page searches. No changes needed to the web page code (it automatically lists all available BLE devices).

Step 3.3: Flash the ESP32 Code

1. Connect the **ESP32** to your computer with a USB cable
2. Open Arduino IDE, click **File** → **Open** → select [ESP32_Bluetooth_Chat/ESP32_Bluetooth_Chat.ino](#)
3. Click **Tools** → **Board** → select your ESP32 dev board (e.g., [ESP32 Dev Module](#))
4. Click **Tools** → **Port** → select the COM port corresponding to your ESP32
5. Click the **Upload** button (arrow icon) to start flashing
6. Wait for flashing to complete — you'll see [Hard resetting via RTS pin...](#) indicating success

Step 3.4: Open the Serial Monitor

1. Open the serial monitor in Arduino IDE ([Ctrl+Shift+M](#))
2. Change the **baud rate** to **115200** (very important! Otherwise you'll see garbled text)
3. ⚠ **Critical setting:** Set the **line ending** to **"Newline"** or **"Both NL & CR"**
4. Press the **EN button** (reset button) on the ESP32
5. Observe the serial output and compare with the "Expected Output" section below

Step 3.5: Open the Web Page

Method 1: Double-click to open (recommended to try first)

Double-click [web/index.html](#) and it will open with your default browser. If your default browser is Chrome or Edge, it usually works directly.

Method 2: Open with a local server (most stable)

If Method 1 doesn't work (some Chrome versions restrict Bluetooth permissions for [file://](#) protocol), start a local server using Python:

1. Open a terminal / command prompt
2. Navigate to the [web](#) folder:
3. Start the local server:

```
1 | python -m http.server 8000
```

4. Open <http://localhost:8000> in Chrome or Edge



💡 Don't have Python? You can also use VS Code's "Live Server" extension — right-click [index.html](#) → "Open with Live Server".

✓ Step 4: Verify the Results

[Back to Elegoo](#)

4.1 ESP32 Serial Expected Output

```
1 | =====
2 | ESP32 BLE Chat — Starting up...
3 | =====
4 | [BLE] BLE device name: ESP32_BLE_Chat
5 | [BLE] Advertising started. Open the web page to connect.
6 |
7 | [BLE] Type a message and press Enter to send to the web page.
8 | [BLE] Commands: /ledon /ledoff /help
```

- ▶ Seeing [BLE device name: ESP32_BLE_Chat](#) → BLE initialized successfully ✓
- ▶ Seeing [Advertising started](#) → Advertising started, can be discovered by web page ✓

4.2 Web Page Connection

1. Open [index.html](#) in Chrome / Edge
2. Click the "Connect Bluetooth" button
3. The browser pops up a Bluetooth device selection window, select [ESP32_BLE_Chat](#)
4. The web page shows "Connected to ESP32_BLE_Chat", and the status indicator turns green
5. ESP32 serial will show: [\[BLE\] >>> Web client connected!](#)

4.3 Two-Way Chat Effect

Type a message on the web page and press Enter or click "Send":

- ▶ Web page displays: [\[14:30:25\] Hello ESP32!](#) (green, right-aligned)
- ▶ ESP32 serial displays: [\[BLE\] Received: Hello ESP32!](#)

Type a message on the ESP32 serial and press Enter:

- ▶ ESP32 serial displays: [\[BLE\] Sent: Hello web page!](#)
- ▶ Web page displays: [\[14:30:35\] Hello web page!](#) (blue, left-aligned)

4.4 LED Command Control

Method 1: Click web page buttons

- ▶ Click "Turn LED On" button → ESP32's GPIO2 LED lights up
- ▶ Click "Turn LED Off" button → ESP32's GPIO2 LED turns off

The web page displays the sent command, and ESP32 serial shows:

```
1 | [BLE] >>> Web command: LED turned ON
```

ESP32 also sends a confirmation message via Bluetooth, and the web page displays:

```
1 | [14:31:00] [ESP32] LED turned ON
```

Method 2: Type commands directly in the web page input box

Type [/ledon](#) or [/ledoff](#) in the web page input box and press Enter — the effect is the same as clicking the buttons.

Method 3: Type commands in the ESP32 serial port

Type [/ledon](#) or [/ledoff](#) in the ESP32 serial monitor — ESP32's local LED is also controlled, and the web page is notified via Bluetooth.

Get help: Type [/help](#) on either end to display the available command list.

4.5 Disconnect and Reconnect

Close the web page tab or click the "Disconnect" button. ESP32 serial will show:

```
1 | [BLE] <<< Web client disconnected. Restarting advertising...
```

ESP32 automatically restarts advertising. Simply open the web page again and click "Connect Bluetooth" to reconnect. [Back to Elegoo](#)

🙄 Step 5: Troubleshooting Common Issues



💡 **Newcomers, read this first:** 90% of issues are covered below.

Q1: Serial monitor shows garbled text or blank?

A: Incorrect baud rate.

- ▶ Check the baud rate in the bottom-right corner of the serial monitor — **it must be 115200**
- ▶ After changing, press the EN (reset) button on the ESP32

Q2: No response after typing messages in the serial port?

A: Incorrect line ending setting.

- ▶ Set the bottom-right corner of the serial monitor to **"Newline"** or **"Both NL & CR"**
- ▶ It cannot be "No line ending"

Q3: Compilation error — BLE library not found?

A: ESP32 core version is too old or wrong board selected.

1. Update the [esp32](#) core to the latest version in Arduino IDE's "Board Manager"
2. **Tools** → **Board** → select [ESP32 Dev Module](#) (do not select ESP32-S2)
3. The BLE library is built into the ESP32 core — no additional installation needed

Q4: Web page doesn't show device selection window after clicking "Connect Bluetooth"?

A: Browser doesn't support Web Bluetooth or permission denied.

1. 🔴 **Verify browser:** Must use **Chrome** or **Edge** — Firefox / Safari are not supported
2. 🔴 **Verify protocol:** If `file://` doesn't work, use a local server (<http://localhost:8000>)
3. 🔴 **Verify computer has Bluetooth:** Desktop computers may not have a Bluetooth module — need an external USB Bluetooth adapter
4. 🔴 **Verify Bluetooth is enabled:** Windows Settings → Bluetooth & other devices → ensure Bluetooth is turned on

Q5: ESP32_BLE_Chat not visible in the device selection window?

A: ESP32 is not advertising or computer can't find it.

1. 🔴 **Verify ESP32 is flashed and running:**
 - ▶ Serial should show [Advertising started](#)
2. 🔴 **Press EN button to restart ESP32:** Sometimes advertising state is abnormal — restarting fixes it
3. 🔴 **Move closer:** Place ESP32 within 1 meter of the computer
4. 🔴 **Check Bluetooth name:** Confirm `BLEDevice::init("ESP32_BLE_Chat")` in the code hasn't been modified incorrectly

Q6: Connected successfully but can't receive messages?

A: Possibly characteristic subscription failed.

1. **Verify web page shows "Connected":** Status indicator should be green
2. **Verify ESP32 serial shows connection:** Should have `>>> Web client connected!`
3. **Refresh web page and retry:** Disconnect → refresh page → reconnect
4. **Check UUID consistency:** SERVICE_UUID and other UUIDs must be exactly the same in both web page and ESP32 code

Q7: Messages sent from ESP32 serial not received by web page?

A: Possibly not connected or timing issue.

1. **Verify connected:** ESP32 serial should show [Web client connected](#)
2. **Check serial input:** Did you press Enter after typing the message? Is the line ending correct (Q2)?
3. **Message when not connected:** If not connected, ESP32 will show [Not connected. Open the web page first.](#)

Q8: LED doesn't light up?

[Back to Elegoo](#)

A: Check the following:

1. **Verify command was sent:** Web page should show [\[ESP32\] LED turned ON](#)
2. **Verify LED pin:** Code uses `LED_PIN = 2` (GPIO2) — most ESP32 boards have the onboard LED on this pin
3. **Some boards have active-low LEDs:** Some ESP32 boards' LEDs light up at low level — you need to swap [HIGH](#) and [LOW](#)
4. **Observe ESP32 serial:** Should show `>>> Web command: LED turned ON`

Q9: Long messages sent incompletely?

A: BLE default MTU limitation.

- The code already sets `BLEDevice::setMTU(512)` — single message max about 500 bytes
- If messages are longer, they need to be sent in segments
- Chat messages generally won't exceed this limit

Q10: Code changes not taking effect?

A:

1. ESP32 code changes must be re-flashed
2. Web page changes require browser refresh (`Ctrl+F5` for forced refresh)
3. If UUIDs were modified, both ESP32 and web page must be consistent

Troubleshooting Quick Reference (Quick Location)

Symptom	Most Likely Cause	Fastest Solution
Garbled / blank serial	Wrong baud rate	Change to 115200, press EN reset
No response to serial input	Wrong line ending	Change to "Newline" or "Both NL & CR"
Compilation error: BLE not found	ESP32 core too old	Update esp32 core to latest version
Web page can't pop up Bluetooth window	Browser not supported	Switch to Chrome or Edge
Can't find ESP32 device	ESP32 not advertising / Bluetooth off	Restart ESP32, check computer Bluetooth
Connected but can't receive messages	UUID mismatch / subscription failed	Check UUID, refresh and reconnect
LED not lighting	Wrong pin / active-low LED	Confirm GPIO2, try reversing HIGH/LOW
Web page file:// not working	Protocol restriction	Use local server http://localhost:8000 
Changes not taking effect	Not re-flashed / not refreshed	Re-upload + Ctrl+F5 refresh

Step 6: Understand the Project Principles

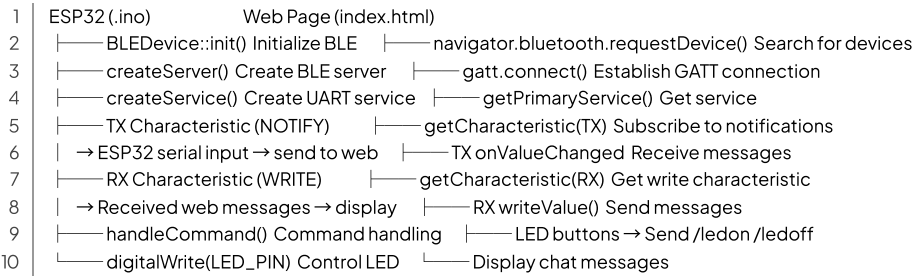


💡 After completing the experiment, understanding the principles behind it will give you twice the result with half the effort!

6.1 Project Architecture: ESP32 + Web Page Division of Labor

This project uses a **single-board + web page** architecture:

Device	Role	Main Responsibilities	Analogy
ESP32	BLE Peripheral	Advertising Bluetooth, waiting for connections, sending/receiving messages, controlling LED	Chat server + controlled device
Computer web page	BLE Central	Searching for devices, establishing connections, sending/receiving messages, sending commands	Chat client + controller



6.2 Why Use BLE Instead of Classic Bluetooth?

This project **must use BLE (Bluetooth Low Energy)** rather than Classic Bluetooth (Classic BT / SPP) because:

Feature	Classic BT (SPP)	BLE	This Project
Web Bluetooth API Support	❌ Not supported	✅ Supported	✅ Must use BLE
Transfer Model	Serial stream	GATT service/characteristic	GATT
Power Consumption	Higher	Very low	BLE
Connection Speed	Slower (pairing)	Fast (direct connection)	BLE
Chip Support	ESP32 (original) only	ESP32 / S3 / C3, etc.	BLE



🔑 **Key reason: Web Bluetooth API (webpage Bluetooth) only supports BLE, not Classic Bluetooth SPP.** If you use BluetoothSerial (Classic Bluetooth), the web page cannot connect. Therefore this project uses BLE's Nordic UART-like service to simulate serial communication.

6.3 What is Nordic UART Service?

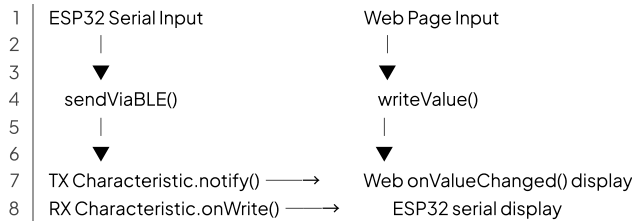
BLE uses the **GATT (Generic Attribute Profile)** protocol for data exchange, with core concepts being "Service" and "Characteristic":

Concept	Analogy	Description
Service	An application	A collection of related functions, identified by UUID
Characteristic	A data item within the application	The actual data-bearing unit with different properties

Nordic UART Service is a de facto BLE serial standard, defined by Nordic Semiconductor, widely used for BLE serial communication:

Role	UUID	Property	Direction
Service	6e400001-...	—	—
TX Characteristic	6e400002-...	Notify	ESP32 → Web page
RX Characteristic	6e400003-...	Write	Web page → ESP32

Data Flow:



💡 **Why use Nordic UART Service?** Because it's widely supported — many BLE serial tools and tutorials use this set of UUIDs, making interoperability convenient.

6.4 Web Bluetooth API Workflow

[Back to Elegoo](#)

The complete process of connecting the web page to ESP32:

```
1 1. navigator.bluetooth.requestDevice({ filters: [{ services: [SERVICE_UUID] }] })
2   ↳ Browser pops up device selection window, user selects ESP32_BLE_Chat
3
4 2. device.gatt.connect()
5   ↳ Establish GATT connection
6
7 3. server.getPrimaryService(SERVICE_UUID)
8   ↳ Get Nordic UART Service
9
10 4. service.getCharacteristic(TX_CHAR_UUID)
11   ↳ Get TX characteristic (ESP32 → Web page)
12
13 5. txCharacteristic.startNotifications()
14   ↳ Subscribe to notifications — data from ESP32 will trigger events
15
16 6. txCharacteristic.addEventListener('characteristicvaluechanged', callback)
17   ↳ Register callback function — automatically called when data is received
18
19 7. service.getCharacteristic(RX_CHAR_UUID)
20   ↳ Get RX characteristic (Web page → ESP32)
21
22 8. rxCharacteristic.writeValue(data)
23   ↳ Send data to ESP32
```



⚠ Security restriction: `requestDevice()` must be triggered by a user gesture (such as a button click) — it cannot be called automatically. This is a browser security policy.

6.5 ESP32 BLE Callback Mechanism

The ESP32 BLE library uses **callback functions** to handle asynchronous events, similar to Classic Bluetooth event callbacks:

Callback	Trigger Timing	Handling in This Project
<code>onConnect()</code>	Web page connects to ESP32	Set <code>deviceConnected = true</code> , print log
<code>onDisconnect()</code>	Web page disconnects	Set <code>deviceConnected = false</code> , restart advertising
<code>onWrite()</code>	Data arrives from web page	Parse message, handle command or display as chat



🔥 Callback vs Polling: Callbacks are "event-driven" — they only execute when there's something to do, wasting no CPU. They are much more efficient than constantly checking in `loop()`.

6.6 Command System Design

This project distinguishes commands from regular messages using the `/` prefix:

Command	Source	Function
<code>/ledon</code>	Web page or serial	Turn on GPIO2 LED
<code>/ledoff</code>	Web page or serial	Turn off GPIO2 LED
<code>/help</code>	Web page or serial	Display help information

Command Processing Flow:

```
1 Message received (web onWrite or serial input)
2 |
3 ▼
4 handleCommand(msg) — Is it a command? —→ Execute LED control / Show help
5 | No
6 ▼
7 Display as regular chat message
```



💡 This "command + regular message" coexistence design is exactly like group chat bot commands or QQ's `/` commands — the same channel can both chat and issue control commands.

[Back to ES900](#)

🔍 Step 7: Code Walkthrough

7.1 Libraries and Key Functions Used in the Project

7.1.1 ESP32–Side Libraries

Library	Source	Description
BLEDevice.h	ESP32 core built-in	BLE device management — initialization, create server/client
BLEServer.h	ESP32 core built-in	BLE server — handle connection/disconnection callbacks
BLEUtils.h	ESP32 core built-in	BLE utility classes
BLE2902.h	ESP32 core built-in	CCC descriptor — required for notify functionality

7.1.2 ESP32 BLE Key Functions

Function/Instruction	Meaning and Purpose
BLEDevice::init(name)	Initialize BLE device , set Bluetooth name
BLEDevice::setMTU(512)	Set MTU size , affects max bytes per transmission
BLEDevice::createServer()	Create BLE server
pServer->setCallbacks(cb)	Register server callback (connect/disconnect events)
pServer->createService(uuid)	Create BLE service
pService->createCharacteristic(uuid, props)	Create characteristic , specify UUID and properties
pTx->addDescriptor(new BLE2902())	Add CCC descriptor , required for notify functionality
pTx->setValue(data)	Set characteristic data
pTx->notify()	Send notification to connected clients
pRx->setCallbacks(cb)	Register characteristic callback (write events)
pCharacteristic->getValue()	Get written data
BLEDevice::startAdvertising()	Start advertising , so clients can discover and connect

7.1.3 Web–Side Web Bluetooth API

Function/Instruction	Meaning and Purpose
navigator.bluetooth.requestDevice(options)	Pop up device selection window , filter by service UUID
device.gatt.connect()	Establish GATT connection
server.getPrimaryService(uuid)	Get specified service
service.getCharacteristic(uuid)	Get specified characteristic
characteristic.startNotifications()	Subscribe to notifications , trigger events when data arrives
characteristic.writeValue(data)	Write data to characteristic

Function/Instruction	Meaning and Purpose	Back to Elegoo
<code>characteristicvaluechanged</code> event	Notification event , triggered when characteristic data changes	

7.2 ESP32 Code Analysis: ESP32_Bluetooth_Chat.ino

7.2.1 Configuration Section

```

1 | #define SERVICE_UUID    "6e400001-b5a3-f393-e0a9-e50e24dcca9e"
2 | #define CHARACTERISTIC_UUID_RX "6e400003-b5a3-f393-e0a9-e50e24dcca9e" //Write (Web -> ESP32)
3 | #define CHARACTERISTIC_UUID_TX "6e400002-b5a3-f393-e0a9-e50e24dcca9e" // Notify (ESP32 -> Web)
4 |
5 | #define LED_PIN 2
6 |
7 | BLEServer *pServer = nullptr;
8 | BLECharacteristic *pTxCharacteristic = nullptr;
9 | bool deviceConnected = false;

```

Runtime Logic: Defines BLE service UUID, LED pin, and state variables.

Key Instruction Analysis:

- ▶ Three UUIDs follow the Nordic UART Service standard — **the web page must use the same UUIDs**
- ▶ `RX` characteristic: Web page writes data here (Web → ESP32)
- ▶ `TX` characteristic: ESP32 sends data via notification (ESP32 → Web)
- ▶ `LED_PIN = 2` : GPIO2, onboard LED
- ▶ `pTxCharacteristic` : Global pointer because `notify()` needs to be called from multiple places
- ▶ `deviceConnected` : Connection state flag, controls whether data is sent

7.2.2 `sendViaBLE()`: Send Message to Web Page

```

1 | void sendViaBLE(const String &msg) {
2 |     if (deviceConnected && pTxCharacteristic != nullptr) {
3 |         pTxCharacteristic->setValue(msg + "\n").c_str());
4 |         pTxCharacteristic->notify();
5 |         delay(10);
6 |     }
7 | }

```

Runtime Logic: Check connection status, write message to TX characteristic, and send notification.

Key Instruction Analysis:

- ▶ `setValue((msg + "\n").c_str())` : Set data to be sent, append `\n` as message separator
- ▶ `notify()` : Push data proactively to subscribed clients
- ▶ `delay(10)` : Give the BLE stack time to send data, avoiding packet loss during continuous sends

7.2.3 `handleCommand()`: Command Processing Function

```

1 | bool handleCommand(const String &msg, const String &source) {
2 |     if (msg == "/ledon") {
3 |         digitalWrite(LED_PIN, HIGH);
4 |         Serial.println("[BLE] >>> " + source + " command: LED turned ON");
5 |         sendViaBLE("[ESP32] LED turned ON");
6 |         return true;
7 |     }
8 |     if (msg == "/ledoff") {
9 |         digitalWrite(LED_PIN, LOW);
10 |        Serial.println("[BLE] >>> " + source + " command: LED turned OFF");
11 |        sendViaBLE("[ESP32] LED turned OFF");
12 |        return true;
13 |    }
14 |    if (msg == "/help") {
15 |        sendViaBLE("[ESP32] Commands: /ledon /ledoff /help");
16 |        return true;
17 |    }
18 |
19 |

```

```

    return false;
}

```

[Back to Elegoo](#)

Runtime Logic: Check if the message is a known command — if so, execute and return `true` .

Key Instruction Analysis:

- `source` parameter: Marks the command source ("Web" or "Local"), making log distinction easier
- `digitalWrite(LED_PIN, HIGH/LOW)` : Control LED on/off
- `sendViaBLE(...)` : Send confirmation message to web page after executing the command
- Return `true` : Tells the caller "this is a command, already handled — don't display as chat message"

7.2.4 BLE Callback Functions

```

1 | class ServerCallbacks : public BLEServerCallbacks {
2 |     void onConnect(BLEServer *pServer) {
3 |         deviceConnected = true;
4 |         Serial.println("[BLE] >>> Web client connected!");
5 |     }
6 |     void onDisconnect(BLEServer *pServer) {
7 |         deviceConnected = false;
8 |         Serial.println("[BLE] <<< Web client disconnected. Restarting advertising...");
9 |         BLEDevice::startAdvertising();
10 |     }
11 | };
12 |
13 | class RxCallbacks : public BLECharacteristicCallbacks {
14 |     void onWrite(BLECharacteristic *pCharacteristic) {
15 |         String rxValue = pCharacteristic->getValue();
16 |         if (rxValue.length() == 0) return;
17 |
18 |         String msg = rxValue;
19 |         msg.trim();
20 |         if (msg.length() == 0) return;
21 |
22 |         if (!handleCommand(msg, "Web")) {
23 |             Serial.print("[BLE] Received: ");
24 |             Serial.println(msg);
25 |         }
26 |     }
27 | };

```

Runtime Logic:

- `ServerCallbacks` : Handle connect/disconnect events, automatically restart advertising on disconnect
- `RxCallbacks` : Handle data from web page — first check if it's a command, if not, display as chat

Key Instruction Analysis:

- `onConnect` / `onDisconnect` : BLE server callbacks, triggered asynchronously
- `onWrite` : Characteristic write callback — triggered every time the web page calls `writeValue()`
- `pCharacteristic->getValue()` : Get the raw bytes written
- `BLEDevice::startAdvertising()` : Restart advertising after disconnect, waiting for next connection

7.2.5 `setup()`: Initialization

```

1 | void setup() {
2 |     Serial.begin(115200);
3 |     pinMode(LED_PIN, OUTPUT);
4 |
5 |     BLEDevice::init("ESP32_BLE_Chat");
6 |     BLEDevice::setMTU(512);
7 |
8 |     pServer = BLEDevice::createServer();
9 |     pServer->setCallbacks(new ServerCallbacks());
10 |
11 |     BLEService *pService = pServer->createService(SERVICE_UUID);
12 |
13 |     // TX characteristic (notify)
14 |     pTxCharacteristic = pService->createCharacteristic(
15 |

```

```

15 | CHARACTERISTIC_UUID_TX, BLECharacteristic::PROPERTY_NOTIFY);
16 | pTxCharacteristic->addDescriptor(new BLE2902());
17 |
18 | // RX characteristic (write)
19 | BLECharacteristic *pRxCharacteristic = pService->createCharacteristic(
20 |     CHARACTERISTIC_UUID_RX,
21 |     BLECharacteristic::PROPERTY_WRITE | BLECharacteristic::PROPERTY_WRITE_NR);
22 | pRxCharacteristic->setCallbacks(new RxCallbacks());
23 |
24 | pService->start();
25 | BLEDevice::startAdvertising();
26 | }

```

[Back to Elegoo](#)

Runtime Logic:

1. Initialize serial and LED
2. Initialize BLE device, set name and MTU
3. Create server, register connection callbacks
4. Create Nordic UART service and two characteristics
5. Start service and advertising

Key Instruction Analysis:

- ▶ `BLEDevice::setMTU(512)` : Increase MTU, allowing more bytes per single transmission
- ▶ `PROPERTY_NOTIFY` : TX characteristic property, allows ESP32 to proactively push data
- ▶ `BLE2902()` : CCC descriptor — clients subscribe to notifications through it
- ▶ `PROPERTY_WRITE | PROPERTY_WRITE_NR` : RX characteristic supports both acknowledged write and unacknowledged write
- ▶ `pService->start()` : Start the service so clients can discover it
- ▶ `startAdvertising()` : Start advertising so clients can find it

7.2.6 loop(): Main Loop

```

1 | void loop() {
2 |     if (Serial.available()) {
3 |         String msg = Serial.readStringUntil('\n');
4 |         msg.trim();
5 |         if (msg.length() > 0) {
6 |             if (!handleCommand(msg, "Local")) {
7 |                 if (deviceConnected) {
8 |                     sendViaBLE(msg);
9 |                     Serial.print("[BLE] Sent: ");
10 |                     Serial.println(msg);
11 |                 } else {
12 |                     Serial.println("[BLE] Not connected. Open the web page first.");
13 |                 }
14 |             }
15 |         }
16 |     }
17 |     delay(10);
18 | }

```

Runtime Logic:

1. Check serial input
2. First determine if it's a local command (like `/ledon`) — if so, execute directly
3. If not a command, send it to the web page via BLE
4. When not connected, prompt the user to open the web page first

Key Instruction Analysis:

- ▶ `Serial.available()` : **Non-blocking** check for serial input
- ▶ `handleCommand(msg, "Local")` : Local commands can also control LED without requiring web page connection
- ▶ `sendViaBLE(msg)` : Send regular messages via BLE notification to the web page
- ▶ All BLE data reception is handled in callbacks; `loop()` only handles serial input

7.3 Web Page Code Analysis: index.html

7.3.1 Connection Flow

```

1 | connectBtn.addEventListener('click', async () => {
2 |   bluetoothDevice = await navigator.bluetooth.requestDevice({
3 |     filters: [{ services: [SERVICE_UUID]}]
4 |   });
5 |   bluetoothDevice.addEventListener('gattserverdisconnected', onDisconnected);
6 |
7 |   const server = await bluetoothDevice.gatt.connect();
8 |   const service = await server.getPrimaryService(SERVICE_UUID);
9 |
10 |   txCharacteristic = await service.getCharacteristic(TX_CHAR_UUID);
11 |   rxCharacteristic = await service.getCharacteristic(RX_CHAR_UUID);
12 |
13 |   await txCharacteristic.startNotifications();
14 |   txCharacteristic.addEventListener('characteristicvaluechanged', onValueChanged);
15 | });

```

Runtime Logic:

1. `requestDevice()` : Pop up device selection window, filter by service UUID
2. `gatt.connect()` : Establish GATT connection
3. Get service and two characteristics
4. Subscribe to TX characteristic notifications

Key Instruction Analysis:

- `filters: [{ services: [SERVICE_UUID]}]` : Only show devices containing this service, filtering out irrelevant Bluetooth devices
- `await` : Web Bluetooth API is entirely Promise-based — must use `async/await` or `.then()`
- `startNotifications()` : Subscribe to notifications — every time ESP32 calls `notify()` , an event will be triggered

7.3.2 Sending Messages

```

1 | async function sendMessage(msg) {
2 |   const data = new TextEncoder().encode(msg + '\n');
3 |   await rxCharacteristic.writeValue(data);
4 |   displayMessage(msg, 'sent');
5 | }

```

Runtime Logic: Encode the message as a byte array, write it to the RX characteristic — ESP32's `onWrite` callback will receive it.

Key Instruction Analysis:

- `TextEncoder().encode()` : Encode string to UTF-8 byte array
- `writeValue(data)` : Write data to characteristic — triggers `onWrite` callback on ESP32 side
- Append `\n` : Consistent with ESP32 convention, used as message separator

7.3.3 Receiving Messages

```

1 | function onValueChanged(event) {
2 |   const value = event.target.value;
3 |   let text = "";
4 |   for (let i = 0; i < value.byteLength; i++) {
5 |     text += String.fromCharCode(value.getUint8(i));
6 |   }
7 |   receiveBuffer += text;
8 |
9 |   let idx;
10 |   while ((idx = receiveBuffer.indexOf('\n')) >= 0) {
11 |     const msg = receiveBuffer.substring(0, idx).trim();
12 |     receiveBuffer = receiveBuffer.substring(idx + 1);
13 |     if (msg.length > 0) {
14 |       displayMessage(msg, 'received');
15 |     }
16 |   }
17 | }

```

Runtime Logic:

1. Convert received bytes to string

2. Append to receive buffer
3. Split by `\n`, each complete message is displayed as one chat entry

[Back to Elegoo](#)

Key Instruction Analysis:

- `event.target.value`: DataView object containing received bytes
- `getUint8(i)`: Read byte by byte
- Buffer + line-based splitting: Prevents incorrect concatenation when a message is split across multiple notifications

7.4 Customization Guide

7.4.1 Changing Bluetooth Name

ESP32 side:

```
1 | BLEDevice::init("your_bluetooth_name");
```

No changes needed on the web page — the device selection window will automatically list the discovered device names.

7.4.2 Changing LED Pin

```
1 | #define LED_PIN 2 // Change to your desired pin
```

Common onboard LED pins for ESP32 development boards:

- ESP32 Dev Module: GPIO2
- ESP32-CAM: Usually no onboard LED, can use GPIO4
- Some boards: GPIO4 / GPIO13, etc.

7.4.3 Adding New Control Commands

Add an `if` branch in the `handleCommand()` function of the ESP32 code:

```
1 | if(msg == "/blink"){ // Blink 3 times
2 |   for(int i = 0; i < 3; i++){
3 |     digitalWrite(LED_PIN, HIGH); delay(200);
4 |     digitalWrite(LED_PIN, LOW); delay(200);
5 |   }
6 |   sendViaBLE("[ESP32] Blink done");
7 |   return true;
8 | }
```

Add a corresponding button on the web page:

```
1 | <button id="blinkBtn" disabled>Blink LED</button>
```

```
1 | document.getElementById('blinkBtn').addEventListener('click', () => sendMessage('/blink'));
```

7.4.4 Modifying the Web Page UI

All web page styles are in the `<style>` tag, and all logic is in the `<script>` tag. You can freely modify:

- Colors, fonts, layout
- Add more control buttons
- Add message timestamp format
- Add message history storage

[← Dual Board WIFI](#)

[Dual Board Bluetooth→](#)